

Institute of Information Technology
Jahangirnagar University
Savar, Dhaka-1342



Lab Report : 8

Lab Title: Process Scheduling Algorithms

Course Code: ICT-3102
Course Name: Operating System Lab

Submitted to

Mehrin Anannya
Assistant Professor
Institute of Information Technology
Jahangirnagar University

Submitted By

Md. Shaon Khan
ID: 1984
Session: 2022-2023
Batch: 52

Submission Date: 15th July 2026

1. Objective

This lab experiment covers three CPU process scheduling algorithms used by operating systems: Longest Job First (non-preemptive), Longest Remaining Time First (preemptive), and Round Robin (preemptive). For each process, the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) are calculated, along with the average waiting time and average turnaround time. The lab also builds practical skill in coding a scheduling algorithm and in comparing how each algorithm affects overall system performance.

2. Introduction to the Algorithms

2.1 Longest Job First (LJF) — Non-Preemptive

Longest Job First is a non-preemptive scheduling algorithm. Among all processes that have already arrived in the ready queue, the process with the largest burst time is chosen to run next. Once a process starts, it runs until it finishes, without interruption. Only processes that have arrived by the current time are considered when choosing the next process.

Algorithm steps:

- Sort the processes in increasing order of Arrival Time.
- Among all processes that have arrived by the current time, choose the one with the highest Burst Time.
- Run the selected process to completion, without preemption.
- Repeat until all processes have finished.

2.2 Longest Remaining Time First (LRTF) — Preemptive

Longest Remaining Time First is the preemptive version of LJF. At every unit of time, the scheduler checks all arrived processes and runs the one with the largest remaining burst time. If a new process arrives with more remaining time than the process currently running, the CPU switches to the new process. This continues until every process has finished.

Algorithm steps:

- Sort the processes in increasing order of Arrival Time.
- At each time unit, select the process with the greatest remaining burst time among those that have arrived.
- Run the selected process for one unit of time and reduce its remaining time by one.
- Repeat this selection at every time unit, allowing preemption, until all processes finish.

2.3 Round Robin (RR) — Preemptive

Round Robin gives each process in the ready queue a fixed time slice, called the Time Quantum (TQ), and cycles through the processes in order. If a process does not finish within its quantum, its remaining burst time is carried over and it is placed back at the end of the queue. If it finishes, it is removed from the queue. Any process that arrives during execution is added to the ready queue and considered in later cycles.

Algorithm steps:

- Sort all processes in increasing order of Arrival Time.
- Assign a fixed Time Quantum (TQ) for execution.
- Starting from the first process in the ready queue, run it for at most TQ units.
- If the process finishes within the quantum, remove it from the queue. Otherwise, subtract the time used from its remaining burst time and place it at the end of the queue.
- Add any newly arrived processes to the ready queue as they arrive.
- Repeat until all processes are finished, then compute the Average Waiting Time and Average Turnaround Time.

3. Source Code and Results

The same input set was used to test all three algorithms, so that their performance could be compared directly:

Process	AT	BT	CT	TAT	WT
P1	0	5	-	-	-
P2	1	3	-	-	-
P3	2	8	-	-	-
P4	3	6	-	-	-

3.1 Longest Job First (LJF)

The following Python program implements the non-preemptive LJF scheduling algorithm:

```
n = int(input("Enter number of processes: "))
pid = []
at = []
bt = []
ct = [0] * n
tat = [0] * n
wt = [0] * n
done = [False] * n
avgWT = 0
avgTAT = 0

for i in range(n):
    pid.append(i + 1)
    arrival, burst = map(int, input(f"Enter Arrival Time and Burst Time for P{i+1}: ").split())
    at.append(arrival)
    bt.append(burst)

completed = 0
current = 0
while completed < n:
    idx = -1
    maxBT = -1
```

```

for i in range(n):
    if not done[i] and at[i] <= current:
        if bt[i] > maxBT:
            maxBT = bt[i]
        idx = i
    if idx == -1:
        current += 1
        continue
    current += bt[idx]
    ct[idx] = current
    tat[idx] = ct[idx] - at[idx]
    wt[idx] = tat[idx] - bt[idx]
    avgWT += wt[idx]
    avgTAT += tat[idx]
    done[idx] = True
    completed += 1

print("\nProcess\tAT\tBT\tCT\tTAT\tWT")
for i in range(n):
    print(f"P{pid[i]}\t{at[i]}\t{bt[i]}\t{ct[i]}\t{tat[i]}\t{wt[i]}")

print(f"\nAverage Waiting Time = {avgWT/n:.2f}")
print(f"Average Turnaround Time = {avgTAT/n:.2f}")

```

Output for Arrival Times (0, 1, 2, 3) and Burst Times (5, 3, 8, 6):

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	22	21	18
P3	2	8	13	11	3
P4	3	6	19	16	10

```

Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 5
Enter Arrival Time and Burst Time for P2: 1 3
Enter Arrival Time and Burst Time for P3: 2 8
Enter Arrival Time and Burst Time for P4: 3 6

Process AT      BT      CT      TAT      WT
P1      0       5       5       5       0
P2      1       3      22      21      18
P3      2       8      13      11       3
P4      3       6      19      16      10

Average Waiting Time = 7.75
Average Turnaround Time = 13.25

```

Fig 3.1: Console output of the LJF program

Average Waiting Time = 7.75 ms Average Turnaround Time = 13.25 ms

3.2 Longest Remaining Time First (LRTF)

The following Python program implements the preemptive LRTF scheduling algorithm:

```
n = int(input("Enter number of processes: "))
pid = []
at = []
bt = []
rt = []
ct = [0] * n
tat = [0] * n
wt = [0] * n
avgWT = 0
avgTAT = 0

for i in range(n):
    pid.append(i + 1)
    arrival, burst = map(int, input(f"Enter Arrival Time and Burst Time for P{i+1}: ").split())
    at.append(arrival)
    bt.append(burst)
    rt.append(burst)

completed = 0
current = 0
while completed < n:
    idx = -1
    maxRT = -1
    # Find the process with the longest remaining time
    for i in range(n):
        if rt[i] > 0 and at[i] <= current:
            if rt[i] > maxRT:
                maxRT = rt[i]
                idx = i
    # If no process has arrived yet
    if idx == -1:
        current += 1
        continue
    # Run the selected process for 1 unit of time
    rt[idx] -= 1
    current += 1
    # If the process finishes
    if rt[idx] == 0:
        ct[idx] = current
        tat[idx] = ct[idx] - at[idx]
        wt[idx] = tat[idx] - bt[idx]
        avgWT += wt[idx]
        avgTAT += tat[idx]
        completed += 1

print("\nProcess\tat\tbt\tCT\tTAT\tWT")
for i in range(n):
    print(f"P{pid[i]}\t{at[i]}\t{bt[i]}\t{ct[i]}\t{tat[i]}\t{wt[i]}")

print(f"\nAverage Waiting Time = {avgWT / n:.2f}")
print(f"Average Turnaround Time = {avgTAT / n:.2f}")
```

Output for the same set of processes:

Process	AT	BT	CT	TAT	WT
P1	0	5	19	19	14
P2	1	3	20	19	16
P3	2	8	21	19	11
P4	3	6	22	19	13

```

Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 5
Enter Arrival Time and Burst Time for P2: 1 3
Enter Arrival Time and Burst Time for P3: 2 8
Enter Arrival Time and Burst Time for P4: 3 6

Process AT      BT      CT      TAT      WT
P1      0       5       19       19       14
P2      1       3       20       19       16
P3      2       8       21       19       11
P4      3       6       22       19       13

Average Waiting Time = 13.50

```

Fig 3.2: Console output of the LRTF program

Average Waiting Time = 13.50 ms Average Turnaround Time = 19.00 ms

3.3 Round Robin (RR)

The following Python program implements the Round Robin scheduling algorithm with a time quantum entered by the user:

```

n = int(input("Enter number of processes: "))
pid = []
at = []
bt = []
rt = []
ct = [0] * n
tat = [0] * n
wt = [0] * n
avgWT = 0
avgTAT = 0

for i in range(n):
    pid.append(i + 1)
    arrival, burst = map(int, input(f"Enter Arrival Time and Burst Time for P{i+1}: ").split())
    at.append(arrival)
    bt.append(burst)
    rt.append(burst)

```

```

tq = int(input("Enter Time Quantum: "))

completed = 0
current = 0
while completed < n:
    done = True
    for i in range(n):
        if rt[i] > 0 and at[i] <= current:
            done = False
            if rt[i] > tq:
                current += tq
                rt[i] -= tq
            else:
                current += rt[i]
                rt[i] = 0
                ct[i] = current
                tat[i] = ct[i] - at[i]
                wt[i] = tat[i] - bt[i]
                avgWT += wt[i]
                avgTAT += tat[i]
                completed += 1
    if done:
        current += 1 # CPU idle

print("\nProcess\tat\tbt\tCT\tTAT\tWT")
for i in range(n):
    print(f"P{pid[i]}\t{at[i]}\t{bt[i]}\t{ct[i]}\t{tat[i]}\t{wt[i]}")

print(f"\nAverage Waiting Time = {avgWT / n:.2f}")
print(f"Average Turnaround Time = {avgTAT / n:.2f}")

```

Output for the same set of processes with Time Quantum = 2:

Process	AT	BT	CT	TAT	WT
P1	0	5	16	16	11
P2	1	3	11	10	7
P3	2	8	22	20	12
P4	3	6	20	17	11

```

Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 5
Enter Arrival Time and Burst Time for P2: 1 3
Enter Arrival Time and Burst Time for P3: 2 8
Enter Arrival Time and Burst Time for P4: 3 6
Enter Time Quantum: 2

Process AT    BT    CT    TAT    WT
P1      0      5     16     16     11
P2      1      3     11     10      7
P3      2      8     22     20     12
P4      3      6     20     17     11

Average Waiting Time = 10.25
Average Turnaround Time = 15.75

```

Fig 3.3: Console output of the Round Robin program

Average Waiting Time = 10.25 ms Average Turnaround Time = 15.75 ms

4. Comparison of Results

The table below summarizes the average waiting time and average turnaround time obtained from each algorithm for the identical input set (AT = 0, 1, 2, 3 and BT = 5, 3, 8, 6):

Algorithm	Avg. Waiting Time (ms)	Avg. Turnaround Time (ms)
LJF (non-preemptive)	7.75	13.25
LRTF (preemptive)	13.50	19.00
Round Robin (TQ = 2)	10.25	15.75

For this input set, LJF gives the lowest average waiting and turnaround time, since only four short processes are involved and the largest jobs also happen to finish efficiently in this arrival pattern. LRTF performs worst here because repeatedly switching to the process with the most remaining work forces shorter jobs, such as P2, to wait a long time before finishing. Round Robin falls in between: the fixed quantum shares the CPU fairly among all processes, avoiding the long waits seen in LRTF, while still adding some extra waiting time from context switching compared to LJF.

5. Discussion

Longest Job First keeps the number of context switches low and works well when process burst times are known in advance, but it is unfair to shorter processes, which may wait a long time if longer jobs keep arriving. LRTF, the preemptive version, adapts to arriving processes but is even more likely to delay short processes, because a newly arrived long process can repeatedly interrupt whichever job is currently running. Round Robin avoids this problem entirely, since every process gets a turn within one queue cycle, which makes it well suited to time-sharing systems. Its performance does depend on the choice of time quantum: too small a quantum increases overhead from context switching, while too large a quantum makes Round Robin behave like First Come First Served.

6. Conclusion

In this lab, three CPU scheduling algorithms — Longest Job First, Longest Remaining Time First, and Round Robin — were implemented and tested on identical process sets. Comparing the completion, turnaround, and waiting times from each program showed how the choice of scheduling policy directly affects process wait times and overall responsiveness. The experiment also highlighted the trade-off between throughput-oriented, non-preemptive strategies like LJF and fairness-oriented, preemptive strategies like Round Robin, and why real operating systems favour quantum-based approaches such as Round Robin for interactive, time-shared workloads.

Institute of Information Technology
Jahangirnagar University
Savar, Dhaka-1342



Lab Report : 9

Lab Title: Banker's Algorithm

Course Code: ICT-3102
Course Name: Operating System Lab

Submitted to

Mehrin Anannya
Assistant Professor
Institute of Information Technology
Jahangirnagar University

Submitted By

Md. Shaon Khan
ID: 1984
Session: 2022-2023
Batch: 52

Submission Date: 15th July 2026

1. Objective

This session covers the Banker's Algorithm, a classic deadlock avoidance algorithm used in operating systems. The lab covers the theory behind the algorithm and includes a practical implementation to check whether a system is in a safe state.

2. Introduction to the Algorithm

2.1 Definition of Banker's Algorithm

The Banker's Algorithm is a deadlock avoidance algorithm. It simulates the allocation of resources to processes to check for a safe state. A state is "safe" if the system can allocate resources to each process, up to its maximum requested amount, in some order, while still avoiding deadlock. The algorithm is named after a bank that allocates credit to customers while making sure it never reaches a state where it cannot satisfy all outstanding credit requests.

2.2 Why the Banker's Algorithm?

Deadlock is a serious problem in operating systems, where two or more processes wait indefinitely, each holding a resource that another waiting process needs. Deadlock detection allows deadlocks to occur and then recovers from them. Deadlock avoidance instead prevents the system from ever entering an unsafe state. By running a safety check before granting each resource request, the algorithm makes sure the system always stays in a state where a safe sequence of execution exists.

2.3 Key Data Structures

- Available: a vector of length m (number of resource types) showing the number of available instances of each resource type.
- Max: an $n \times m$ matrix (n = number of processes) showing the maximum demand of each process.
- Allocation: an $n \times m$ matrix showing the number of resources of each type currently allocated to each process.
- Need: an $n \times m$ matrix showing the remaining resource need of each process, calculated as $\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$.

2.4 Steps of the Safety Algorithm

- Initialize $\text{Work} = \text{Available}$, and $\text{Finish}[i] = \text{false}$ for all processes i .
- Find an index i such that $\text{Finish}[i] == \text{false}$ and $\text{Need}[i] \leq \text{Work}$.
- If such a process is found, set $\text{Work} = \text{Work} + \text{Allocation}[i]$ and $\text{Finish}[i] = \text{true}$.
- Repeat steps 2–3 until no such process can be found.
- If $\text{Finish}[i] == \text{true}$ for all processes, the state is safe and the order of execution is a safe sequence. Otherwise, the system is unsafe.

3. Source Code and Results

The following Python program implements the Banker's Algorithm and its safety check:

```
def input_matrix(rows, cols, name):
    print(f"\nEnter the '{name}' matrix ({rows} x {cols}):")
    matrix = []
    for i in range(rows):
        row = list(map(int, input(f"P{i}: ").split()))
        matrix.append(row)
    return matrix

def calculate_need(max_matrix, allocation, n, m):
    need = [[max_matrix[i][j] - allocation[i][j] for j in range(m)] for i in range(n)]
    return need

def is_safe(available, max_matrix, allocation, need, n, m):
    work = available.copy()
    finish = [False] * n
    safe_sequence = []
    count = 0
    while count < n:
        found = False
        for i in range(n):
            if not finish[i]:
                # Check if Need[i] <= Work
                if all(need[i][j] <= work[j] for j in range(m)):
                    # Allocate resources, simulate process completion
                    for j in range(m):
                        work[j] += allocation[i][j]
                    safe_sequence.append(i)
                    finish[i] = True
                    found = True
                    count += 1
        if not found:
            # No process could be allocated -> unsafe state
            return False, []
    return True, safe_sequence

def main():
    n = int(input("Enter the number of processes: "))
    m = int(input("Enter the number of resource types: "))
    print(f"\nEnter the 'Available' vector ({m} resources):")
    available = list(map(int, input().split()))
    max_matrix = input_matrix(n, m, "Max")
    allocation = input_matrix(n, m, "Allocation")
    need = calculate_need(max_matrix, allocation, n, m)
    safe, sequence = is_safe(available, max_matrix, allocation, need, n, m)
    if safe:
        print("\nSystem is in a safe state.")
        print("Safe Sequence is:", " -> ".join(f"P{p}" for p in sequence))
    else:
        print("\nSystem is in an unsafe state. No safe sequence exists.")

if __name__ == "__main__":
```

```
main()
```

The program was tested with a system of 5 processes (P0–P4) and 3 resource types (A, B, C), with Available = (3, 3, 2):

Process	Allocation (A,B,C)	Max (A,B,C)	Need (A,B,C)
P0	0, 1, 0	7, 5, 3	7, 4, 3
P1	2, 0, 0	3, 2, 2	1, 2, 2
P2	3, 0, 2	9, 0, 2	6, 0, 0
P3	2, 1, 1	2, 2, 2	0, 1, 1
P4	0, 0, 2	4, 3, 3	4, 3, 1

```
Enter the number of processes: 5
Enter the number of resource types: 3

Enter the 'Available' vector (3 resources):
3 3 2

Enter the 'Max' matrix (5 x 3):
P0: 7 5 3
P1: 3 2 2
P2: 9 0 2
P3: 2 2 2
P4: 4 3 3

Enter the 'Allocation' matrix (5 x 3):
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1
P4: 0 0 2

System is in a safe state.
Safe Sequence is: P1 -> P3 -> P4 -> P0 -> P2
```

Fig 9.1: Console output of the Banker's Algorithm program

The program correctly identifies the system as being in a safe state, and produces the safe sequence P1 → P3 → P4 → P0 → P2, which matches the manual calculation of the safety algorithm.

4. Discussion

The Banker's Algorithm shows how an operating system can avoid deadlock proactively, rather than detecting and recovering from it afterward. By maintaining the Available, Max, Allocation, and Need data structures, and repeatedly searching for a process whose remaining need can be met by the currently available resources, the algorithm builds a safe sequence one process at a time. If at any point no remaining process can have its need satisfied by the available resources, the system is declared unsafe, since granting further requests could lead to a circular wait. The main trade-off of this approach is the overhead of running the safety check before every resource request, but it guarantees that the system never enters a deadlocked state.

5. Conclusion

In this lab, the Banker's Algorithm was implemented in Python and verified against a standard 5-process, 3-resource-type example. The program correctly determined that the system was in a safe state and produced the safe sequence $P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$, consistent with the manual trace of the safety algorithm. The experiment reinforced the practical value of deadlock avoidance, showing how a resource manager can check the safety of a state before committing to an allocation, which is essential in systems where multiple processes compete for limited, shared resources.